

Q1)

1. Design:

```
1  module state_2_Array ();
2
3      bit [11:0] my_array [4];
4
5      initial begin
6          my_array[0] = 12'h012;
7          my_array[1] = 12'h345;
8          my_array[2] = 12'h678;
9          my_array[3] = 12'h9AB;
10
11          $display("For Loop:");
12          for(int i = 0; i < $size(my_array); i++)begin
13              $display("bits [5:4] of Element
14 :%b | All Data:%b", i, my_array[i][5:4], my_array[i]);
15              $display("\n Foreach Loop:");
16              foreach (my_array[i]) begin
17                  $display("bits [5:4] of Element
18 :%b | All Data:%b", i, my_array[i][5:4], my_array[i]);
19              end
20
21      endmodule
```

2. Questasim Output:

```
# For Loop:
# bits [5:4] of Element 0:01 | All Data:000000010010
# bits [5:4] of Element 1:00 | All Data:001101000101
# bits [5:4] of Element 2:11 | All Data:011001111000
# bits [5:4] of Element 3:10 | All Data:100110101011
#
# Foreach Loop:
# bits [5:4] of Element 0:01 | All Data:000000010010
# bits [5:4] of Element 1:00 | All Data:001101000101
# bits [5:4] of Element 2:11 | All Data:011001111000
# bits [5:4] of Element 3:10 | All Data:100110101011
```

Q2)

1. Design:

```
1  module ALU_4_bit (  
2      input  clk,  
3      input  reset,  
4      input  [1:0] Opcode,    // The opcode  
5      input  signed [3:0] A, // Input data A in 2's complement  
6      input  signed [3:0] B, // Input data B in 2's complement  
7  
8      output reg signed [4:0] C // ALU output in 2's complement  
9  
10     );  
11  
12     reg signed [4:0]    Alu_out; // ALU output in 2's complement  
13  
14     localparam        Add          = 2'b00; // A + B  
15     localparam        Sub          = 2'b01; // A - B  
16     localparam        Not_A        = 2'b10; // ~A  
17     localparam        ReductionOR_B = 2'b11; // |B  
18  
19     // Do the operation  
20     always @* begin  
21         case (Opcode)  
22             Add:          Alu_out = A + B;  
23             Sub:          Alu_out = A - B;  
24             Not_A:        Alu_out = ~A;  
25             ReductionOR_B: Alu_out = |B;  
26             default: Alu_out = 5'b0;  
27         endcase  
28     end // always @*  
29  
30     // Register output C  
31     always @(posedge clk or posedge reset) begin  
32         if (reset)  
33             C <= 5'b0;  
34         else  
35             C <= Alu_out;  
36     end  
37  
38 endmodule  
39
```

2. Testbench Code

Test Bench:

```
1  import ALU_pkg::*;
2
3  module ALU_tb();
4
5      logic clk;
6      logic reset;
7      OPCODE_e Opcode; // The opcode
8      logic signed [3:0] A; // Input data A in 2's complement
9      logic signed [3:0] B; // Input data B in 2's complement
10
11      logic signed [4:0] C; // ALU output in 2's complement
12
13      ALU_Inputs inputs = new;
14
15      int error_count, correct_count;
16
17      ALU_4_bit DUT (.*);
18
19      initial begin
20          clk = 0;
21          forever
22              #1 clk = ~clk;
23      end
24
25      initial begin
26
27          reset_assert();
28
29          repeat(30)begin
30              assert(inputs.randomize());
31              reset = inputs.reset;
32              Opcode = inputs Opcode;
33              A = inputs.A;
34              B = inputs.B;
35              if(reset)begin
36                  check_result(0);
37              end
38              else begin
39                  case(Opcode)
40                      ADD: begin
41                          check_result(A + B);
42                      end
43                      SUB: begin
44                          check_result(A - B);
45                      end
46                      NOT_A: begin
47                          check_result(~A);
48                      end
49                      ReductionOR_B: begin
50                          if(|B)
51                              check_result(1);
52                          else
53                              check_result(0);
54                      end
55                      default: begin
56                          $display("Error ---> Invalid Opcode");
57                          $stop;
58                      end
59                  endcase
60              end
61          end
62          $display("Total Tests:
63 | Correct: %d", correct_count);
64 | Error: %d", error_count);
65 ", correct_count + error_count, correct_count, error_count);
66      task reset_assert();
67          reset = 1;
68          check_result(0);
69          reset = ~reset;
70      endtask
71
72      task check_result(input logic signed [4:0] C_expected);
73          @(negedge clk);
74          if(C != C_expected) begin
75              $display("Expected:%d | DUT:%d <----- Error", C_expected, C);
76              error_count++;
77              $stop;
78          end
79          else begin
80              $display("Expected:%d | DUT:%d", C_expected, C);
81              correct_count++;
82          end
83      endtask
84
85  endmodule
```

Package:

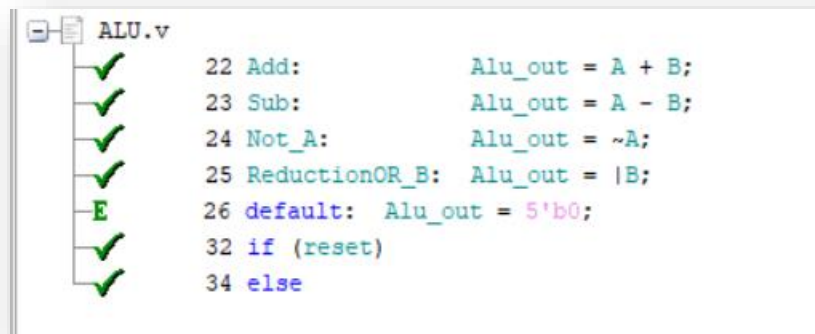
```
1 package ALU_pkg;
2
3     typedef enum logic [1:0] {ADD, SUB, NOT_A, ReductionOR_B} OPCODE_e;
4
5     class ALU_Inputs;
6
7         rand logic          reset;
8         rand OPCODE_e       Opcode;    // The opcode
9         rand logic signed [3:0] A; // Input data A in 2's complement
10        rand logic signed [3:0] B; // Input data B in 2's complement
11
12        function new(logic reset = 0, logic signed A = 0, B = 0, OPCODE_e Opcode = ADD);
13
14            this.reset = reset;
15            this.A = A;
16            this.B = B;
17            this.Opcode = Opcode;
18
19        endfunction //new()
20
21        constraint c_reset{
22            reset dist{0:/95, 1:/5};
23        }
24    endclass //ALU_Inputs
25
26
27
28
29 endpackage
```

3. Do File:

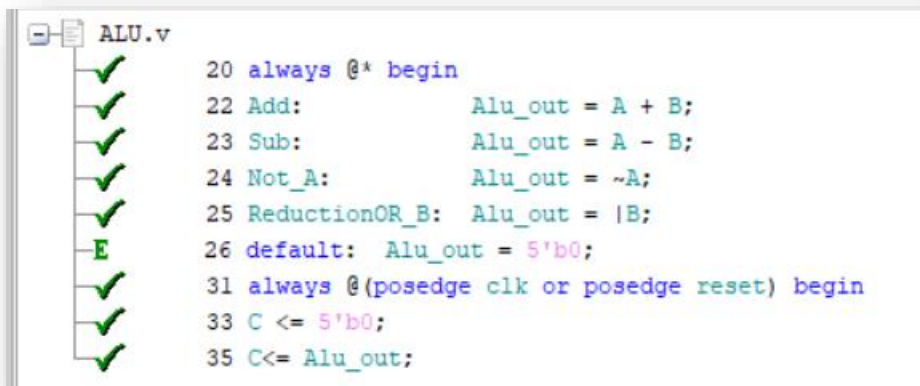
```
1  vlib work
2  vlog ALU.v ALU_tb.sv ALU_pkg.sv +cover -covercells
3  vsim -voptargs=+acc work.ALU_tb -cover
4  do wave.do
5  coverage save ALU_tb.ucdb -onexit -du work.ALU
6  run -all
7  coverage exclude -src ALU.v -line 26 -code s
8  coverage exclude -src ALU.v -line 26 -code b
9  #quit -sim
10 #vcover report ALS_tb.ucdb -details -annotate -all -output coverage_rpt.txt
```

4. Code Coverage:

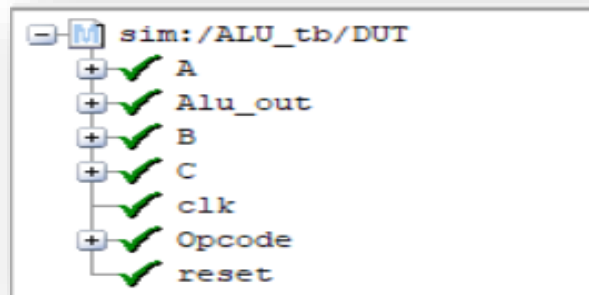
Branch Coverage:



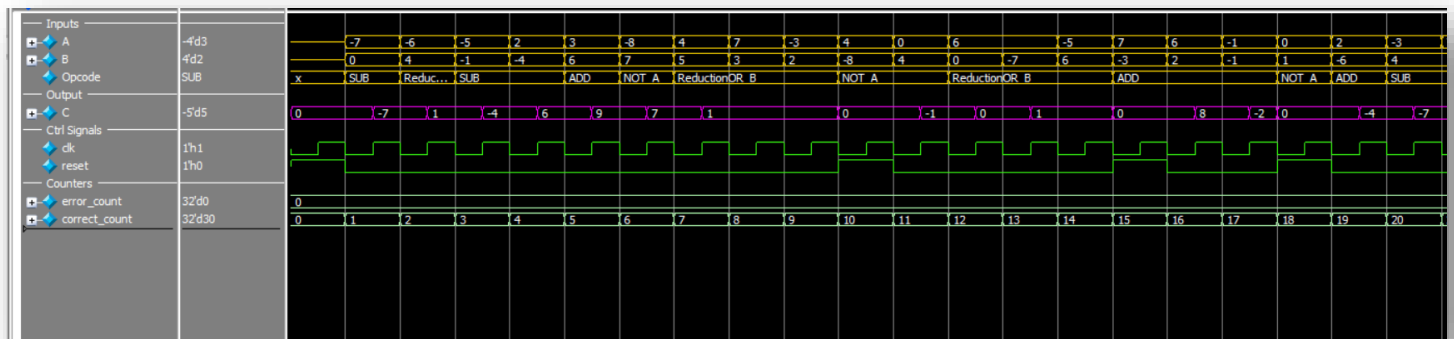
Statement Coverage:



Toggle Coverage:



5. Questasim Waveform



6. Coverage Report:

Branch Coverage:

```
=====
== Instance: /\ALU_tb#DUT
== Design Unit: work.ALU_4_bit
=====

Branch Coverage:
  Enabled Coverage      Bins      Hits      Misses      Coverage
  -----
  Branches              6         6         0      100.00%

=====Branch Details=====

Branch Coverage for instance /\ALU_tb#DUT

  Line      Item      Count      Source
  ----      -
  File ALU.v
  -----CASE Branch-----
  21              30      Count coming in to CASE
  22              8      Add:      Alu_out = A + B;
  23              8      Sub:      Alu_out = A - B;
  24              5      Not_A:    Alu_out = ~A;
  25              9      ReductionOR_B: Alu_out = |B;

Branch totals: 4 hits of 4 branches = 100.00%
```

Statement Coverage:

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	8	8	0	100.00%

=====Statement Details=====

Toggle & Total Coverage:

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	44	44	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\ALU_tb#DUT --

Node	1H->0L	0L->1H	"Coverage"
A[0-3]	1	1	100.00
Alu_out[4-0]	1	1	100.00
B[0-3]	1	1	100.00
C[4-0]	1	1	100.00
Opcode[0-1]	1	1	100.00
clk	1	1	100.00
reset	1	1	100.00

Total Node Count = 22
Toggled Node Count = 22
Untoggled Node Count = 0

Toggle Coverage = 100.00% (44 of 44 bins)

Total Coverage By Instance (filtered view): 100.00%

Q3)

1. Design

```
 8 module FSM_010(clk, rst, x, y, users_count);
 9     parameter IDLE = 2'b00;
10     parameter ZERO = 2'b01;
11     parameter ONE  = 2'b10;
12     parameter STORE = 2'b11;
13
14     input clk, rst, x;
15     output y;
16     output reg [9:0] users_count;
17
18     reg [1:0] cs, ns;
19
20     always @(*) begin
21         case (cs)
22             IDLE:
23                 if (x)
24                     ns = IDLE;
25                 else
26                     ns = ZERO;
27             ZERO:
28                 if (x)
29                     ns = ONE;
30                 else
31                     ns = ZERO;
32             ONE:
33                 if (x)
34                     ns = IDLE;
35                 else
36                     ns = STORE;
37             STORE:
38                 if (x)
39                     ns = IDLE;
40                 else
41                     ns = ZERO;
42             default: ns = IDLE;
43         endcase
44     end
45
46     always @(posedge clk or posedge rst) begin
47         if(rst) begin
48             cs <= IDLE;
49         end
50         else begin
51             cs <= ns;
52         end
53     end
54
55     always @(posedge clk or posedge rst) begin
56         if(rst) begin
57             users_count <= 0;
58         end
59         else begin
60             if (cs == STORE)
61                 users_count <= users_count + 1;
62             end
63         end
64
65         assign y = (cs == STORE)? 1:0;
66
67     endmodule
```

2. Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functional Checking
FSM_1	When the reset is asserted, the output counter value should be low	Directed at the start of the sim, then randomized with constraint that drive the reset to be off most of the simulation time.		A Golden Model in the testbench to make sure the output is correct
FSM_2	Assigning Values to X input	Randomized Stimulation		A Golden Model in the testbench to make sure the output is correct

3. Test Bench:

Package:

```
1  package FSM_010_pkg;
2
3      typedef enum logic [1:0] {IDLE, ZERO, ONE, STORE} state_e;
4
5      class fsm_transaction;
6
7          rand logic      x, rst, y_exp;
8          rand logic [9:0] user_count_exp;
9
10         constraint c_x{
11             x dist{0:/67, 1:/33};
12         }
13
14         constraint c_rst{
15             rst dist{0:/95, 1:/5};
16         }
17
18         function new(logic x = 0, rst = 0, y = 0, logic [9:0] user_count_exp = 0);
19
20             this.x = x;
21             this.rst = rst;
22             this.y_exp = y;
23             this.user_count_exp = user_count_exp;
24
25         endfunction
26
27     endclass
28
29 endpackage
```

Test Bench:

```
1 import FSM_010_pkg::*;
2
3 module FSM_010_tb();
4
5     logic clk, rst, x;
6     logic y_dut, y_golden;
7     logic [9:0] users_count_dut, users_count_golden;
8
9     fsm_transaction inputs = new;
10
11     int error_count, correct_count;
12
13     FSM_010 dut (.x(x), .clk(clk), .rst(rst), .y(y_dut), .users_count(users_count_dut));
14     Sequence_detect Golden (.x(x), .clk(clk), .rst(rst), .y(y_golden), .count(users_count_golden));
15
16     initial begin
17         clk = 0;
18         forever
19             #1 clk = ~clk;
20     end
21
22     initial begin
23         error_count = 0; correct_count = 0;
24         //FSM_1
25         reset_assert();
26
27         //FSM_2
28         repeat(30)begin
29             assert(inputs.randomize());
30             rst = inputs.rst;
31             x = inputs.x;
32             check_result();
33         end
34
35         //Extra Tests for 100% coverage
36         rst = 0; x = 0; check_result(); //0
37         rst = 0; x = 1; check_result(); //10
38         rst = 0; x = 0; check_result(); //010
39         rst = 0; x = 1; check_result(); //1010
40
41         $display("Total Tests:
42 | Corres$stop$ts:
43 | Eendr Test:
44 ", correct_count + error_count, correct_count, error_count);
45         task reset_assert();
46             rst = 1;
47             check_result;
48             rst = ~rst;
49         endtask
50
51         task check_result();
52             @(negedge clk);
53             if(y_dut != y_golden || users_count_dut != users_count_golden) begin
54                 error_count++;
55                 $display("Expected: Y:
56 , Count: $stop;
57 | DUT: %nd
58 , Count:else begin
59 <---Error", $display0;Eendr$edout$it_golden, y_dut, users_count_dut);
60 , Count: correct_count++;
61 | DUT: %nd
62 , Count$task
63 ", y_golden, users_count_golden, y_dut, users_count_dut);
64     endmodule
```

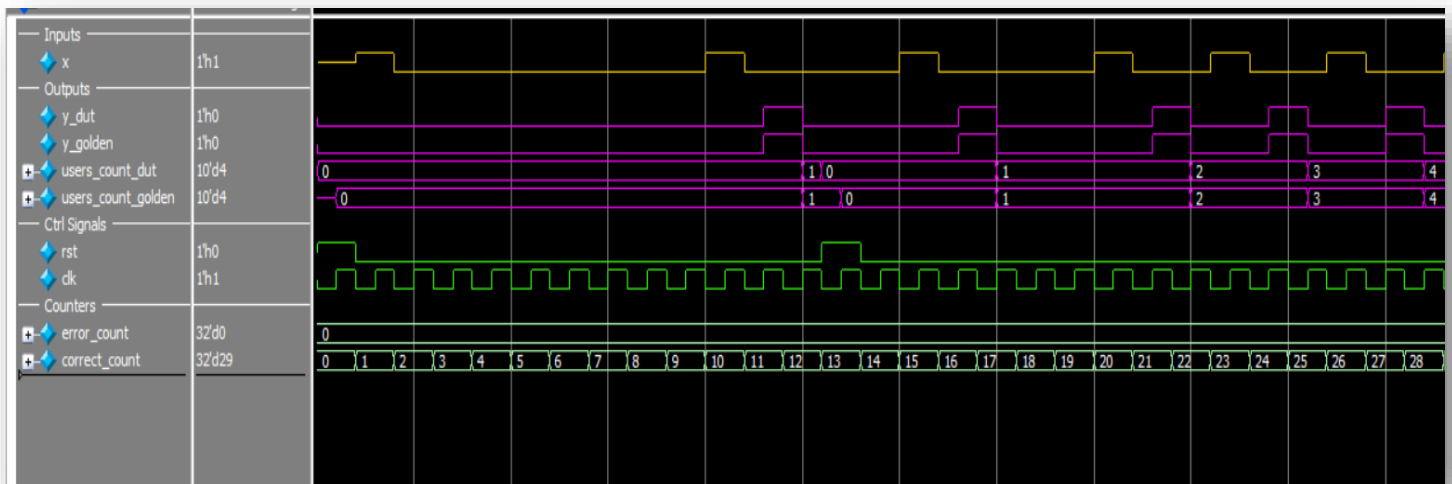
Golden Model:

```
1  module Sequence_detect #(
2      parameter IDLE  = 2'b00,
3      parameter ZERO  = 2'b01,
4      parameter ONE   = 2'b10,
5      parameter STORE = 2'b11
6  ) (
7      input          x,
8      input          clk,
9      input          rst, //Active High Asynch
10     output reg      y,
11     output reg [9:0] count
12 );
13
14 reg [1:0] state, next_state;
15
16 //current state logic
17 always @(posedge clk or posedge rst) begin
18     if(rst) begin
19         state <= IDLE;
20     end
21     else
22         state <= next_state;
23 end
24
25 //next state logic
26 always @(*) begin
27     case (state)
28     IDLE:
29         if(x == 0)
30             next_state = ZERO;
31         else
32             next_state = IDLE;
33     ZERO:
34         if(x == 1)
35             next_state = ONE;
36         else
37             next_state = ZERO;
38     ONE:
39         if(x == 0)
40             next_state = STORE;
41         else
42             next_state = IDLE;
43     STORE:
44         if(x == 0)
45             next_state = ZERO;
46         else
47             next_state = IDLE;
48     default: next_state = IDLE;
49     endcase
50
51 end
52
53 //output logic
54 always @(*)begin
55     y = (state == STORE) ? 1'b1 : 1'b0;
56 end
57
58 always @(posedge clk)begin
59     if(rst)
60         count <= 0;
61     else if(state == STORE)
62         count <= count + 1;
63 end
64
65 endmodule
```

4. Do File

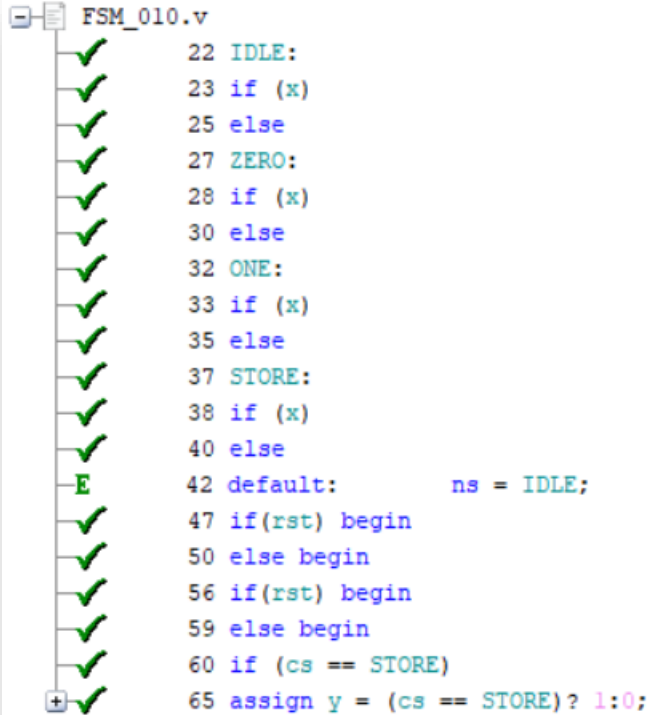
```
1 vlib work
2 vlog FSM_010.v FSM_010_Golden.v FSM_010_pkg.sv FSM_010_tb.sv +cover -covercells
3 vsim -voptargs=+acc work.FSM_010_tb -cover
4 do wave.do
5 coverage save FSM_010_tb.ucdb -onexit -du work.FSM_010
6 run -all
7 coverage exclude -du FSM_010 -togglenode {users_count[2]}
8 coverage exclude -du FSM_010 -togglenode {users_count[3]}
9 coverage exclude -du FSM_010 -togglenode {users_count[4]}
10 coverage exclude -du FSM_010 -togglenode {users_count[5]}
11 coverage exclude -du FSM_010 -togglenode {users_count[6]}
12 coverage exclude -du FSM_010 -togglenode {users_count[7]}
13 coverage exclude -du FSM_010 -togglenode {users_count[8]}
14 coverage exclude -du FSM_010 -togglenode {users_count[9]}
15 coverage exclude -src FSM_010.v -line 42 -code s
16 coverage exclude -src FSM_010.v -line 42 -code b
17 quit -sim
18 vcover report FSM_010_tb.ucdb -details -annotate -all -output coverage_rpt.txt
```

5. QuestaSim Waveform:



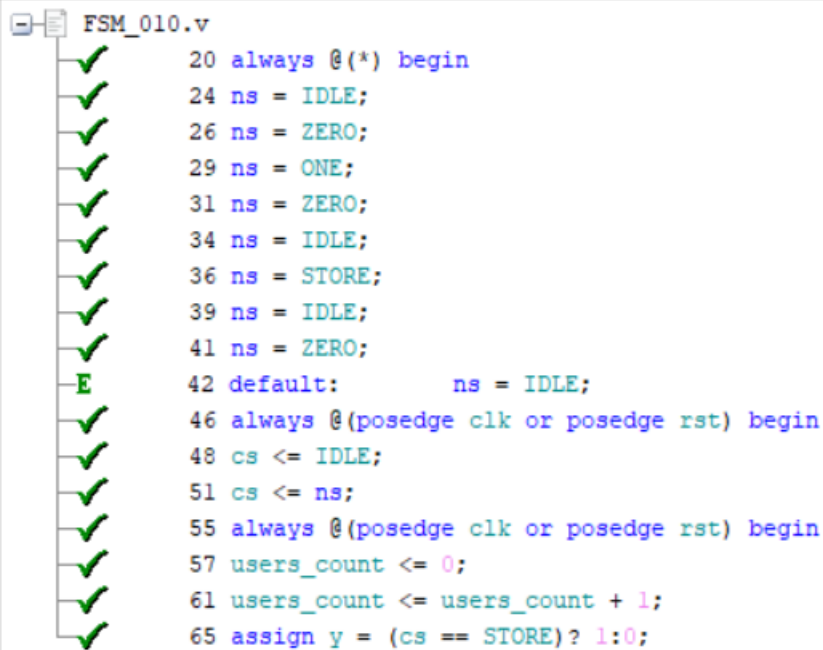
6. Code Coverage:

Branch Coverage:



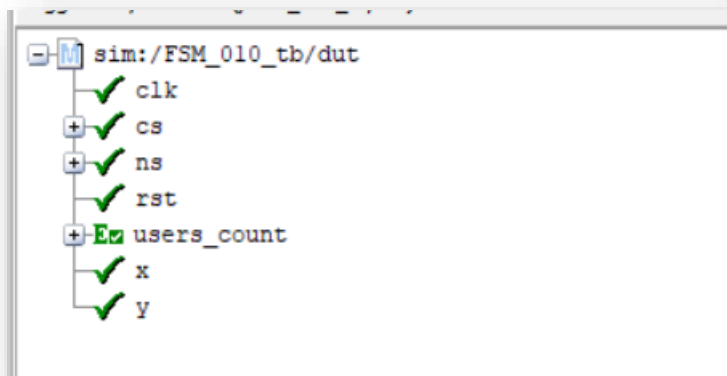
```
FSM_010.v
22 IDLE:
23 if (x)
25 else
27 ZERO:
28 if (x)
30 else
32 ONE:
33 if (x)
35 else
37 STORE:
38 if (x)
40 else
42 default:      ns = IDLE;
47 if(rst) begin
50 else begin
56 if(rst) begin
59 else begin
60 if (cs == STORE)
65 assign y = (cs == STORE)? 1:0;
```

Statement Coverage:



```
FSM_010.v
20 always @(*) begin
24 ns = IDLE;
26 ns = ZERO;
29 ns = ONE;
31 ns = ZERO;
34 ns = IDLE;
36 ns = STORE;
39 ns = IDLE;
41 ns = ZERO;
42 default:      ns = IDLE;
46 always @(posedge clk or posedge rst) begin
48 cs <= IDLE;
51 cs <= ns;
55 always @(posedge clk or posedge rst) begin
57 users_count <= 0;
61 users_count <= users_count + 1;
65 assign y = (cs == STORE)? 1:0;
```

Toggle Coverage:



Note: Users_count[9:2] are excluded to reduce the number of test

7. Coverage Report:

Branch Coverage:

```
=====
=== Instance: /\FSM_010_tb#dut
=== Design Unit: work.FSM_010
=====
Branch Coverage:
  Enabled Coverage          Bins    Hits    Misses  Coverage
  -----
  Branches                  20      20       0   100.00%
=====Branch Details=====
```

FSM Coverage:

```
FSM Coverage:
  Enabled Coverage          Bins    Hits    Misses  Coverage
  -----
  FSM States                 4       4       0   100.00%
  FSM Transitions            7       6       1    85.71%
=====FSM Details=====
```

Statement Converge:

Statement Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage

Statements	16	16	0	100.00%

Toggle & Total Coverage:

Toggle Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage

Toggles	20	20	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\FSM_010_tb#dut --

Node	1H->0L	0L->1H	"Coverage"

clk	1	1	100.00
cs[1-0]	1	1	100.00
ns[1-0]	1	1	100.00
rst	1	1	100.00
users_count[1-0]	1	1	100.00
x	1	1	100.00
y	1	1	100.00

Total Node Count = 10
Toggled Node Count = 10
Untoggled Node Count = 0

Toggle Coverage = 100.00% (20 of 20 bins)

Total Coverage By Instance (filtered view): 97.61%

Note: not 100% coverage due to FSM transitions